

Cygnus — Santiago — Assignment Jul-08-2026

Assignment handed out in the 2026-07-08 meeting. Corresponds to **Track 3 — Intermediate C#** of the Cygnus program. See the full program in [cygnus-program.md](#).

Summary of topics to develop

- **Blocks 3.1–3.6 — Intermediate C#:** LINQ, dictionaries and hash sets, NuGet packages, manual multithreading, async/await basics, and a finishing mini-project. Each block introduces one concept, has a guided exercise, and a deliverable.
- **One continuous project:** the whole track builds a single **CLI weather app**. Block 3.1 creates the console app; every block after it extends the *same* project. Keep everything in one repository and commit after each block.
- **Track 3 Challenges (3.A–3.C):** further extensions of the finished weather app, each added on its own branch (weather report by condition, parallel refresh with threads, retry with backoff).

Work in C# with **dotnet** console projects. Each deliverable should be a running program committed via the branch → PR → merge flow already practiced in Track 0.

Block 3.1 — LINQ

Objective: The learner creates the project and queries/transforms collections declaratively instead of with manual loops.

Concepts: `IEnumerable<T>`, method-syntax LINQ (`.Where`, `.Select`, `.OrderBy`, `.OrderByDescending`, `.GroupBy`, `.First/.FirstOrDefault`, `.Any`, `.All`, `.Count`, `.Sum`, `.Average`, `.Max`, `.Min`), lambda expressions (`x => x.TempC`), deferred execution, `.ToList()`.

Guided exercise — Start the weather app and query readings:

1. Create the project as a **console app**: run `dotnet new console -o weather-cli`, then open the `weather-cli` folder in VS Code (`File → Open Folder`). This is the project every later block builds on.
2. Define the data type. Add a **record** (a small immutable data holder) above your top-level code that represents one day's weather: `record DailyReading(DateOnly Date, string Condition, double TempC);`
3. Seed sample data: build a `List<DailyReading>` with ~10 hardcoded days for one city, mixing conditions like `"Sunny"`, `"Rainy"`, `"Cloudy"` and varied temperatures so the queries below have something to work with.
4. Ask the user for a temperature (read it and `double.Parse` it), then use `.Where(r => r.TempC > threshold)` to list every warmer day.
5. Use `.OrderByDescending(r => r.TempC)` to sort the readings, then `.Take(3)` to print the 3 hottest days.
6. Use `.GroupBy(r => r.Condition)`; loop over the resulting groups and, for each, print the condition name, its `.Count()`, and its `.Average(r => r.TempC)`.
7. Pick one of the queries above and rewrite it as a manual **foreach** loop, to feel the difference against LINQ.

Deliverable: Running `weather-cli` console app that filters, sorts, and groups the sample readings using LINQ.

Block 3.2 — Dictionaries and HashSets

Objective: The learner extends the app to hold many cities using key-based and set collections for fast lookups and uniqueness.

Concepts: `Dictionary<TKey, TValue>` (`[]` indexer, `.TryGetValue`, `.ContainsKey`, `.Keys`, `.Values`, iterating `KeyValuePair`), `HashSet<T>` (`.Add`, `.Contains`, uniqueness), when to use a dictionary vs a list.

Guided exercise — Store readings per city:

1. Continue in the `weather-cli` project from Block 3.1.
2. Replace the single list with a `Dictionary<string, List<DailyReading>>` keyed by city name; seed two or three cities.
3. Prompt the user for a city and use `.TryGetValue` to look up its readings (print a friendly message if not found).
4. Reuse the Block 3.1 LINQ queries against the selected city's readings.
5. Build a `HashSet<string>` of every distinct `Condition` across all cities and print how many unique conditions exist.

Deliverable: The weather app now stores multiple cities in a dictionary and reports the set of unique conditions.

Block 3.3 — NuGet packages

Objective: The learner installs a real external library from NuGet — using both the UI and the CLI — and uses it to persist the app's data.

The external package used here is `Newtonsoft.Json` (Json.NET), the most-downloaded package on NuGet. It is truly external (unlike `System.Text.Json`, which ships inside the SDK), so it is a genuine example of pulling in a third-party dependency.

Concepts: what NuGet is, the NuGet gallery (nuget.org), installing a package via the C# Dev Kit UI vs `dotnet add package`, how a `PackageReference` lands in the `.csproj`, using a library namespace, `Newtonsoft.Json` (`JsonConvert.SerializeObject` / `DeserializeObject`), formatting (`Formatting.Indented`), reading/writing JSON files, inspecting packages with `dotnet list package`.

Guided exercise — Save and load the cities as JSON:

1. Continue in the `weather-cli` project; you will persist the cities dictionary from Block 3.2.
2. **Install the package with the NuGet UI (C# Dev Kit):**
 1. Open the Solution Explorer view in VS Code (from the C# Dev Kit).
 2. Right-click the project node and choose **Add NuGet Package** (or open the command palette with `Ctrl+Shift+P` and run **.NET: Add NuGet Package**).
 3. Type `Newtonsoft.Json` in the search box and select it from the results.
 4. Pick the latest stable version and confirm — VS Code adds a `PackageReference` and restores it.
 5. Open the `.csproj` file and confirm the `<PackageReference Include="Newtonsoft.Json" Version="..." />` line is there.

3. **(Alternative) Install the same package from the CLI** to see both paths do the same thing: `dotnet add package Newtonsoft.Json`. Run `dotnet list package` to confirm it is installed.
4. Add `using Newtonsoft.Json;` and serialize the cities dictionary with `JsonConvert.SerializeObject(cities, Formatting.Indented)`, then write it to `weather-data.json`.
5. On startup, if `weather-data.json` exists, read it and deserialize it back with `JsonConvert.DeserializeObject<Dictionary<string, List<DailyReading>>>(json);` otherwise fall back to the seed data.
6. Verify the data round-trips: serialize → file → restart the app → the saved cities load back.

Deliverable: The weather app installs `Newtonsoft.Json` from NuGet and persists its cities to JSON between runs.

Block 3.4 — Manual multithreading

Objective: The learner refreshes several cities "at once" using real threads by hand — and feels the pain that `async/await` later solves.

This block exists so the learner understands *where* `async` came from. Before `async/await`, concurrency meant creating threads yourself, joining them, and protecting shared state. We do that first, on purpose — on the same weather app.

Concepts: `Thread`, `thread.Start()`, `thread.Join()`, running multiple threads, shared mutable state, race conditions, `lock` (mutual exclusion), `Thread.Sleep`, why blocking a thread is expensive.

Guided exercise — Refresh cities on background threads:

1. Continue in the `weather-cli` project.
2. Write a `FetchCity(string city)` method that *simulates* a slow network call: `Thread.Sleep(1000)`, then return a fresh `List<DailyReading>` with a few random temperatures (use `Random.Shared.Next(...)`).
3. Create one thread per city and start them so they run at the same time. Create a thread with a lambda: `var t = new Thread(() => { var readings = FetchCity(city); /* store it */ });` then `t.Start()`; Keep every thread in a `List<Thread>`, then call `.Join()` on each so the program waits for all of them. Time the whole batch and compare it against calling `FetchCity` for each city one after another.
4. Have every thread write its result into the shared cities `Dictionary`. Run it a few times and watch for corrupted or missing entries (a *race condition*) caused by threads writing at the same moment.
5. Add one shared lock object at class level (`private static readonly object _lock = new();`) and wrap each dictionary write in `lock (_lock) { ... }`. Re-run and confirm every city now lands correctly.
6. Note out loud how much code and care this took, and that each thread mostly sat asleep doing nothing useful.

Deliverable: The weather app refreshes multiple cities on parallel threads, showing a race condition fixed with `lock`.

Block 3.5 — Async/await basics

Objective: The learner replaces the manual threads with non-blocking asynchronous code and understands why it is better.

Concepts: `Task`, `Task<T>`, `async` / `await`, `HttpClient.GetStringAsync`, awaiting multiple tasks with `Task.WhenAll`, `Task.Delay` (vs `Thread.Sleep`), why `async` frees the thread instead of blocking it, exceptions in `async` code.

Real API to use — Open-Meteo: free and needs **no API key**, so it drops straight into this exercise.

Two endpoints:

- Find a city's coordinates: `https://geocoding-api.open-meteo.com/v1/search?name=London&count=1`
- Get its daily forecast: `https://api.open-meteo.com/v1/forecast?latitude=51.5&longitude=-0.13&daily=temperature_2m_max,temperature_2m_min&timezone=auto`

The forecast response has parallel arrays under `daily` (`time`, `temperature_2m_max`, `temperature_2m_min`) that map cleanly onto your `DailyReading` list.

Guided exercise — Fetch real data with `async/await`:

1. Continue in the `weather-cli` project. Create one shared `HttpClient` at class level (`private static readonly HttpClient client = new();`) and reuse it for every request — never `new` one per call.
2. Rewrite `FetchCity` as `async Task<List<DailyReading>>`. Inside it: (a) call the geocoding endpoint with `await client.GetStringAsync(geoUrl)` to turn the city name into latitude/longitude, then (b) call the forecast endpoint the same way.
3. Make `Main` `async Task`, `await` a single city fetch, and print the readings.
4. Parse the JSON. The simplest approach with `Newtonsoft.Json` is to `using Newtonsoft.Json.Linq;`, call `JObject.Parse(json)`, and read the arrays by key — e.g. `obj["daily"]["time"]` and `obj["daily"]["temperature_2m_max"]`. Loop by index and build one `DailyReading` per day (pair each date with its max temperature; use a placeholder for `Condition` for now).
5. Refresh several cities at once with `await Task.WhenAll(task1, task2, ...)`; compare the total time against awaiting them one after another.
6. Contrast with Block 3.4: no manual `Thread`, no `Join`, no `lock` — and the thread is not blocked while waiting.

Deliverable: The weather app fetches real forecasts for several cities concurrently using `async/await`.

Guided exercise — Debug the `async` flow in VS Code:

Goal: use the VS Code debugger to watch the `async` fetch run step by step, instead of guessing from `Console` output. Use the `weather-cli` project.

1. **Set a breakpoint:** click in the gutter to the left of the line number on your `await client.GetStringAsync(...)` line — a red dot appears. Set a second breakpoint on the line that prints a deserialized reading.
2. **Start debugging:** press `F5` (choose the C# / .NET configuration if prompted). Execution runs until it hits the first breakpoint and pauses, highlighting the current line.
3. **Inspect values:** hover the mouse over a variable to see its current value; also expand the **Variables** panel (left) to browse locals like `geoUrl`, `city`, and `client`. Add a variable to the **Watch** panel to track

it as you step.

4. **Step over (F10)**: run the current line without entering method calls. Watch the highlighted line advance one line at a time and see values change in the Variables panel.
5. **Step into (F11)**: when stopped on a call to one of *your own* methods (e.g. a parse helper), use Step Into to jump inside that method and follow its execution line by line.
6. **Step out (Shift+F11)**: finish the current method and return to the caller.
7. **Await behaviour**: step over an `await` and notice the debugger pauses until the awaited `Task` completes, then resumes on the next line — confirming the flow is sequential from your code's point of view.
8. **Call stack**: open the **Call Stack** panel to see which method called which, and click frames to inspect their variables.
9. **Continue (F5)**: resume until the next breakpoint or the program ends. Use **Stop** (red square) to end the session early.

Deliverable: Screenshot or short screen recording showing a hit breakpoint with the Variables panel open, plus the learner explaining what step-over, step-into, and the call stack each did.

Block 3.6 — Mini-project: finish the CLI weather app

Objective: The learner polishes everything from Blocks 3.1–3.5 into one finished, reviewed application.

Guided exercise — Ship the weather CLI:

1. Wrap the app in a menu loop: choose a city, refresh (async fetch), view stats, save, or quit.
2. Cache fetched forecasts in the cities `Dictionary` so repeating a city doesn't re-hit the API.
3. Persist the cache to JSON on quit and load it on startup (Block 3.3).
4. Use LINQ to compute and display the min/max/average temperature for the selected city (Block 3.1).
5. Format the output cleanly and handle errors (unknown city, no network, bad JSON) gracefully.

Deliverable: Reviewed PR on GitHub with the finished CLI weather app.

Track 3 Challenges

Challenges extend the **finished CLI weather app** from Block 3.6 to explore a minor concept. Each is a new feature added on a branch of the same project.

Challenge 3.A — Weather report by condition

Uses: Blocks 3.1 + 3.2 · **Explores:** `GroupBy`, `Aggregate`, projections into new shapes · **Difficulty:** ★★

Add a "report" command that groups every stored city's readings by `Condition` and prints a formatted table of day count, average temperature, and hottest day per condition — built entirely with LINQ.

Deliverable: Weather app command that prints a grouped summary report built entirely with LINQ.

Challenge 3.B — Parallel refresh with threads

Uses: Block 3.4 · **Explores:** starting N threads from a loop, collecting results safely · **Difficulty:** ★★★

Add a "refresh all" command that re-fetches every stored city on its own manual **Thread**, joins them all, and safely aggregates the results back into the shared cities dictionary guarded by a **lock** — then compare its timing against the async version from Block 3.5.

Deliverable: Weather app command that refreshes all cities across multiple threads and updates them correctly.

Challenge 3.C — Retry with backoff

Uses: Block 3.5 · **Explores:** **Task.Delay**, exponential retries, catching transient failures · **Difficulty:** ★ ★ ★

Wrap the async Open-Meteo fetch so a failed request retries with an increasing delay (e.g. 1s, 2s, 4s), giving up after a max number of attempts and reporting what happened — so a flaky network no longer crashes the app. Force a failure to test it by pointing at a bad host or an invalid latitude/longitude and confirm the backoff kicks in.

Deliverable: Weather app whose async fetch recovers from transient failures via exponential backoff.